

Pixel Battle — Scripted Combat (Sub-project B)

Implementation Plan

Pixel Battle — Scripted Combat (Sub-project B)

Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (- []) syntax for tracking.

Goal: Replace the RL action source with a ScriptDriver that plays authored conditional-intent fight scripts, add a status-effect system (root / slow / shield / tenacity), and ship a starter script library — so every rendered fight is directed and controlled.

Architecture: A new `script/` package (condition compiler, YAML loader, ScriptDriver) supplies per-tick actions to the existing engine; `_render_fight` is generalised to take any action source. A new `engine/effects.py` adds status effects, applied data-drivenly via an `applies` field on skills and enforced in the engine's physics / hit-resolution.

Tech Stack: Python 3.10, pygame (headless SDL dummy), PyYAML, pytest, ffmpeg.

Design spec: `docs/superpowers/specs/2026-05-22-pixel-battle-scripted-combat-design.md`

File Structure

File	Status	Responsibility
<code>pixel_battle/engine/effects.py</code>	new	<code>StatusEffect</code> + <code>SkillApplies</code> dataclasses; the 4 effect-kind constants
<code>pixel_battle/engine/skill.py</code>	modify	<code>Skill.applies</code> : <code>Optional[SkillApplies]</code> , parsed in <code>from_dict</code>
<code>pixel_battle/engine/character.py</code>	modify	<code>Character.effects</code> list; <code>effect_of / has_effect</code> ; shield-routing in <code>take_damage</code>

File	Status	Responsibility
pixel_battle/engine/battle.py	modify	<code>_update_effects</code> (lifecycle); root/slow in <code>_update_physics</code> ; opponent-effect + tenacity in <code>_resolve_attack_hit</code> ; self- effect in <code>_start_attack_with_kind</code>
pixel_battle/data/characters.json	modify	<code>applies</code> field on 4 skills
pixel_battle/script/__init__.py	new	package marker
pixel_battle/script/conditions.py	new	<code>compile_condition</code> – <code>until</code> string → predicate; <code>ConditionContext</code>
pixel_battle/script/loader.py	new	<code>load_script</code> – YAML → <code>FightScript</code> ; validation
pixel_battle/script/driver.py	new	<code>ScriptDriver</code> – per-tick intent tracking + action emission
pixel_battle/rl/play.py	modify	<code>_render_fight</code> takes an <code>action_source</code> ; <code>_rl_action_source</code> wraps the PPO model; effect VFX
pixel_battle/rl/play_scripted.py	new	scripted render entry point
pixel_battle/data/scripts/*.yaml	new	the starter script library (5 scripts)
pixel_battle/tests/test_*.py	new	one test file per task

No RL retrain: the ScriptDriver replaces RL as the action source. `characters.json` gains data-only `applies` fields. The observation, action space, and reward are untouched.

Action integer map (engine `Discrete(9)` , used throughout): 0 idle, 1 retreat, 2 advance, 3 jump, 4 basic, 5 cd, 6 ultimate, 7 special, 8 kick.

Task 1: `skill.applies` field + effect data

Files: - Create: `pixel_battle/engine/effects.py` - Modify: `pixel_battle/engine/skill.py` - Modify: `pixel_battle/data/characters.json` - Test: `pixel_battle/tests/test_skill_applies.py` (new)

☐ Step 1: Write the failing test

```
# pixel_battle/tests/test_skill_applies.py
"""Skill.applies – data-driven status-effect attachment."""
```

```

from pixel_battle.engine.effects import SkillApplies, ROOT, SHIELD, EFFECT_KINDS
from pixel_battle.engine.skill import Skill

def test_skill_without_applies_is_none():
    s = Skill.from_dict({"id": "x", "type": "basic", "anim": "attack"})
    assert s.applies is None

def test_skill_with_applies_parsing_into_skillapplies():
    s = Skill.from_dict({
        "id": "light_binding", "type": "cooldown", "anim": "light_binding",
        "applies": {"effect": "root", "duration_ms": 1500,
                    "magnitude": 1.0, "target": "opponent"},
    })
    assert isinstance(s.applies, SkillApplies)
    assert s.applies.effect == ROOT
    assert s.applies.duration_ms == 1500
    assert s.applies.target == "opponent"

def test_applies_rejects_unknown_effect():
    import pytest
    with pytest.raises(ValueError):
        Skill.from_dict({"id": "x", "type": "basic", "anim": "attack",
                        "applies": {"effect": "nonsense", "duration_ms": 100,
                                    "magnitude": 1.0, "target": "self"}})

def test_effect_kinds_constant():
    assert EFFECT_KINDS == frozenset({"root", "slow", "shield", "tenacity"})

```

☐ Step 2: Run the test to verify it fails

Run: `python -m pytest pixel_battle/tests/test_skill_applies.py -v` Expected: FAIL —
 ModuleNotFoundError: No module named 'pixel_battle.engine.effects'

☐ Step 3: Create `effects.py`

```

# pixel_battle/engine/effects.py
"""Status effects – root, slow, shield, tenacity."""
from __future__ import annotations

```

```

from dataclasses import dataclass

ROOT = "root"
SLOW = "slow"
SHIELD = "shield"
TENACITY = "tenacity"
EFFECT_KINDS = frozenset({ROOT, SLOW, SHIELD, TENACITY})

_TARGETS = frozenset({"self", "opponent"})

@dataclass
class StatusEffect:
    """A live effect on a character. `magnitude`: slow/tenacity = factor in
    (0, 1); shield = remaining damage pool; root = unused (1.0)."""
    kind: str
    remaining_ms: int
    magnitude: float = 1.0

    def is_expired(self) -> bool:
        return self.remaining_ms <= 0

@dataclass
class SkillApplies:
    """Data-driven status effect a skill attaches when it lands / is cast."""
    effect: str
    duration_ms: int
    magnitude: float
    target: str          # "self" | "opponent"

    @classmethod
    def from_dict(cls, d: dict) -> "SkillApplies":
        effect = d["effect"]
        if effect not in EFFECT_KINDS:
            raise ValueError(f"Unknown status effect: {effect!r}")
        target = d.get("target", "opponent")
        if target not in _TARGETS:
            raise ValueError(f"Unknown applies target: {target!r}")
        return cls(effect=effect, duration_ms=int(d["duration_ms"]),
                    magnitude=float(d.get("magnitude", 1.0)), target=target)

```

☐ Step 4: Add `applies` to `Skill`

In `pixel_battle/engine/skill.py`: add the import and the field. Add to the imports at the top:

```
from typing import Optional
from pixel_battle.engine.effects import SkillApplies
```

Add a field to the `Skill` dataclass (after `vfx`):

```
applies: Optional[SkillApplies] = None
```

In `Skill.from_dict`, parse it — add to the `return cls(...)` call:

```
applies=(SkillApplies.from_dict(d["applies"])
         if d.get("applies") else None),
```

☐ Step 5: Add `applies` data to `characters.json`

In `pixel_battle/data/characters.json`, add an `applies` field to four skills (one per effect kind):

- `lux` → `light_binding` (the cooldown skill): `"applies": {"effect": "root", "duration_ms": 1500, "magnitude": 1.0, "target": "opponent"}`
- `glass_slab` → `shard_scatter` (cooldown): `"applies": {"effect": "slow", "duration_ms": 1800, "magnitude": 0.45, "target": "opponent"}`
- `lux` → `prismatic_barrier` (special): `"applies": {"effect": "shield", "duration_ms": 5000, "magnitude": 25, "target": "self"}`
- `garen` → `courage` (special): `"applies": {"effect": "tenacity", "duration_ms": 5000, "magnitude": 0.5, "target": "self"}`

Add the `applies` key inside each of those skill objects (alongside `id`, `type`, etc.). Leave all other skills unchanged.

☐ Step 6: Run the tests + a JSON-load regression

Run: `python -m pytest pixel_battle/tests/test_skill_applies.py`

`pixel_battle/tests/test_lol_champions.py -v` Expected: PASS — the new tests pass and `characters.json` still loads cleanly for every character.

☐ Step 7: Commit

```
git add pixel_battle/engine/effects.py pixel_battle/engine/skill.py
      pixel_battle/data/characters.json pixel_battle/tests/test_skill_applies.py
git commit -m "feat(pixel-battle/engine): Skill.applies - data-driven status effects"
```

Task 2: StatusEffect on Character + shield-routed damage

Files: - Modify: pixel_battle/engine/character.py - Test:

pixel_battle/tests/test_character_effects.py (new)

character.py current facts (verified): Character is a @dataclass; take_damage is self.hp = max(0, self.hp - amount); reset_physics resets the runtime fields.

☐ Step 1: Write the failing test

```
# pixel_battle/tests/test_character_effects.py
"""Character status-effect storage + shield-routed damage."""
from pixel_battle.engine.character import Character
from pixel_battle.engine.effects import StatusEffect, ROOT, SHIELD

def test_new_character_has_no_effects():
    c = Character.load("garen")
    assert c.effects == []

def test_effect_of_and_has_effect():
    c = Character.load("garen")
    c.effects.append(StatusEffect(kind=ROOT, remaining_ms=1000))
    assert c.has_effect(ROOT) is True
    assert c.effect_of(ROOT).remaining_ms == 1000
    assert c.has_effect(SHIELD) is False
    assert c.effect_of(SHIELD) is None

def test_shield_absorbs_damage_before_hp():
    c = Character.load("garen")
    c.hp = 100
    c.effects.append(StatusEffect(kind=SHIELD, remaining_ms=5000, magnitude=20))
    c.take_damage(8)
    assert c.hp == 100 # fully absorbed
    assert c.effect_of(SHIELD).magnitude == 12

def test_shield_overflow_spills_to_hp_and_shield_drops():
    c = Character.load("garen")
    c.hp = 100
```

```

c.effects.append(StatusEffect(kind=SHIELD, remaining_ms=5000, magnitude=5))
c.take_damage(12)
assert c.hp == 93 # 5 absorbed, 7 to HP
assert c.has_effect(SHIELD) is False # depleted shield removed

```

```

def test_reset_physics_clears_effects():
    c = Character.load("garen")
    c.effects.append(StatusEffect(kind=ROOT, remaining_ms=1000))
    c.reset_physics(initial_x=100.0, facing=1)
    assert c.effects == []

```

☐ Step 2: Run the test to verify it fails

Run: `python -m pytest pixel_battle/tests/test_character_effects.py -v` Expected: FAIL — Character has no effects attribute.

☐ Step 3: Add effects + helpers + shield routing

In `pixel_battle/engine/character.py`:

Add the import at the top:

```

from pixel_battle.engine.effects import StatusEffect, SHIELD

```

Add a field to the `Character` dataclass (after `windup_stun_until_ms`):

```

effects: List["StatusEffect"] = field(default_factory=list)

```

In `reset_physics`, add (next to the other resets):

```

self.effects = []

```

Add two helper methods to the class:

```

def effect_of(self, kind: str):
    for e in self.effects:
        if e.kind == kind:
            return e
    return None

```

```
def has_effect(self, kind: str) -> bool:
    return self.effect_of(kind) is not None
```

Replace `take_damage` with a shield-routed version:

```
def take_damage(self, amount: int) -> None:
    shield = self.effect_of(SHIELD)
    if shield is not None and shield.magnitude > 0:
        absorbed = min(shield.magnitude, amount)
        shield.magnitude -= absorbed
        amount -= absorbed
        if shield.magnitude <= 0:
            self.effects.remove(shield)
    self.hp = max(0, self.hp - amount)
```

☐ Step 4: Run the tests to verify they pass

Run: `python -m pytest pixel_battle/tests/test_character_effects.py -v` Expected: PASS (5/5).

☐ Step 5: Run the engine regression tests

Run: `python -m pytest pixel_battle/tests/test_battle_no_lock.py`
`pixel_battle/tests/test_lol_champions.py -v` Expected: all green.

☐ Step 6: Commit

```
git add pixel_battle/engine/character.py pixel_battle/tests/test_character_effects.py
git commit -m "feat(pixel-battle/engine): Character.effects + shield-routed take_damage"
```

Task 3: Effect lifecycle + root/slow enforcement

Files: - Modify: `pixel_battle/engine/battle.py` - Test:
`pixel_battle/tests/test_effect_enforcement.py` (new)

`battle.py` current facts (verified): `_update_physics(char, dt_ms)` does `char.pos_x = clamp_x(char.pos_x + char.vel_x)` at line 189. `_update_stagger(char, dt_ms)` is at line 222. `tick_ms` calls the per-character updates in its FIGHTING branch.

☐ Step 1: Write the failing test

```
# pixel_battle/tests/test_effect_enforcement.py
"""Effect lifecycle + root/slow enforcement in the engine."""
```

```

import os
os.environ.setdefault("SDL_VIDEODRIVER", "dummy")

from pixel_battle.rl.env import PixelBattleEnv
from pixel_battle.engine.effects import StatusEffect, ROOT, SLOW

def test_update_effects_decrements_and_expires():
    env = PixelBattleEnv(seed=1)
    b = env.battle
    c = env.left
    c.effects.append(StatusEffect(kind=ROOT, remaining_ms=30))
    b._update_effects(c, 16)
    assert c.effect_of(ROOT).remaining_ms == 14
    b._update_effects(c, 16)           # 14 - 16 = expired
    assert c.has_effect(ROOT) is False # expired effect removed

def test_root_forces_zero_velocity():
    env = PixelBattleEnv(seed=1)
    b = env.battle
    c = env.left
    c.vel_x = 5.0
    c.effects.append(StatusEffect(kind=ROOT, remaining_ms=1000))
    start_x = c.pos_x
    b._update_physics(c, 16)
    assert c.pos_x == start_x          # rooted: did not move

def test_slow_scales_velocity():
    env = PixelBattleEnv(seed=1)
    b = env.battle
    fast, slow = env.left, env.right
    fast.pos_x = slow.pos_x = 240.0
    fast.vel_x = slow.vel_x = 6.0
    slow.effects.append(StatusEffect(kind=SLOW, remaining_ms=1000, magnitude=0.5))
    b._update_physics(fast, 16)
    b._update_physics(slow, 16)
    fast_moved = abs(fast.pos_x - 240.0)
    slow_moved = abs(slow.pos_x - 240.0)
    assert slow_moved < fast_moved    # slowed character moved less

```

☐ Step 2: Run the test to verify it fails

Run: `python -m pytest pixel_battle/tests/test_effect_enforcement.py -v` Expected: FAIL — `_update_effects` does not exist.

☐ Step 3: Add `_update_effects` and call it from `tick_ms`

In `pixel_battle/engine/battle.py`, add a new method next to `_update_stagger`:

```
def _update_effects(self, char: Character, dt_ms: int) -> None:
    """Decrement status-effect timers; drop expired effects."""
    for effect in list(char.effects):
        effect.remaining_ms -= dt_ms
        if effect.is_expired():
            char.effects.remove(effect)
```

In `tick_ms`, in the FIGHTING branch where `_update_physics` / `_update_stagger` are already called per character, add a `_update_effects` call for each character right next to the `_update_stagger` calls (read `tick_ms` to find the exact spot — it updates both `self.left` and `self.right`).

☐ Step 4: Enforce root + slow in `_update_physics`

In `_update_physics`, immediately before the line `char.pos_x = clamp_x(char.pos_x + char.vel_x)`, insert root and slow handling:

```
# Status effects: root pins the character; slow scales its movement.
from pixel_battle.engine.effects import ROOT, SLOW
if char.has_effect(ROOT):
    char.vel_x = 0.0
else:
    slow = char.effect_of(SLOW)
    if slow is not None:
        char.vel_x *= slow.magnitude

char.pos_x = clamp_x(char.pos_x + char.vel_x)
```

(Put the `from ... import ROOT, SLOW` at the top of `battle.py` with the other imports instead of inline if `battle.py` has no circular-import issue with `effects.py` — `effects.py` imports nothing from `battle.py`, so a top-level import is clean and preferred.)

☐ Step 5: Run the tests to verify they pass

Run: `python -m pytest pixel_battle/tests/test_effect_enforcement.py -v` Expected: PASS (3/3).

☐ **Step 6: Run the engine regression tests**

```
Run: python -m pytest pixel_battle/tests/test_battle_no_lock.py
pixel_battle/tests/test_hitstop.py -v Expected: all green.
```

☐ Step 7: Commit

```
git add pixel_battle/engine/battle.py pixel_battle/tests/test_effect_enforcement.py
git commit -m "feat(pixel-battle/engine): effect lifecycle + root/slow enforcement"
```

Task 4: Effect application (on hit / on cast) + tenacity

Files: - Modify: pixel_battle/engine/battle.py - Test: pixel_battle/tests/test_effect_application.py (new)

`battle.py` current facts (verified): in `_resolve_attack_hit`, `stagger_ms` is computed at line 304 and the hit fully resolves around lines 304-317. In `_start_attack_with_kind`, the chosen skill is assigned `char.attack_used_kind = skill` at line 520.

- ❑ **Step 1: Write the failing test**

[illegible]

```

atk.attack_used_kind = skill
b._resolve_attack_hit(atk, dfn)
assert dfn.has_effect(ROOT)
assert dfn.effect_of(ROOT).remaining_ms == 1200

def test_tenacity_reduces_applied_stagger():
    env = PixelBattleEnv(seed=1)
    b = env.battle
    atk, dfn = env.left, env.right
    atk.pos_x, dfn.pos_x = 240.0, 300.0
    atk.accuracy = 1.0
    dfn.effects.append(StatusEffect(kind=TENACITY, remaining_ms=5000,
                                    magnitude=0.5))
    b._resolve_attack_hit(atk, dfn)
    from pixel_battle.engine.battle import STAGGER_MS
    # Stagger applied to a tenacious defender is halved.
    assert dfn._stagger_remaining_ms <= STAGGER_MS * 0.5 + 1

def test_self_buff_applies_to_caster_on_cast():
    env = PixelBattleEnv(seed=1)
    b = env.battle
    caster = env.left
    caster.mp = 100
    # Give the caster's special a self-buff applies, then start it.
    sp = caster.skills_of_type(SkillType.SPECIAL)
    assert sp, "character must have a special"
    sp[0].applies = SkillApplies(effect=TENACITY, duration_ms=4000,
                                  magnitude=0.5, target="self")
    b._start_attack_with_kind(caster, env.right, "special")
    assert caster.has_effect(TENACITY)

```

☐ **Step 2: Run the test to verify it fails**

Run: `python -m pytest pixel_battle/tests/test_effect_application.py -v` Expected: FAIL — effects are not applied.

☐ **Step 3: Apply opponent effects + tenacity in `_resolve_attack_hit`**

In `_resolve_attack_hit`, the stagger line is currently:

```
stagger_ms = skill.stagger_ms if skill.stagger_ms > 0 else STAGGER_MS
```

Replace it with a tenacity-aware version:

```
stagger_ms = skill.stagger_ms if skill.stagger_ms > 0 else STAGGER_MS
tenacity = defender.effect_of(TENACITY)
if tenacity is not None:
    stagger_ms = int(stagger_ms * tenacity.magnitude)
```

Then, after the hit fully resolves — directly after `attacker.last_attack_ms = self.elapsed_ms` (line 316, before the `_hitstop_remaining` line is fine too; place it right after that block) — apply an opponent-targeted `applies`:

```
if skill.applies is not None and skill.applies.target == "opponent":
    a = skill.applies
    existing = defender.effect_of(a.effect)
    if existing is not None:
        defender.effects.remove(existing) # refresh, don't stack
    defender.effects.append(StatusEffect(
        kind=a.effect, remaining_ms=a.duration_ms, magnitude=a.magnitude))
```

Add `from pixel_battle.engine.effects import StatusEffect, TENACITY` to `battle.py`'s imports (merge with the `ROOT, SLOW` import from Task 3 → `from pixel_battle.engine.effects import StatusEffect, ROOT, SLOW, TENACITY`).

☐ Step 4: Apply self effects in `_start_attack_with_kind`

In `_start_attack_with_kind`, directly after `char.attack_used_kind = skill` (line 520), add:

```
if skill.applies is not None and skill.applies.target == "self":
    a = skill.applies
    existing = char.effect_of(a.effect)
    if existing is not None:
        char.effects.remove(existing)
    char.effects.append(StatusEffect(
        kind=a.effect, remaining_ms=a.duration_ms, magnitude=a.magnitude))
```

☐ Step 5: Run the tests to verify they pass

Run: `python -m pytest pixel_battle/tests/test_effect_application.py -v` Expected: PASS (3/3).

☐ Step 6: Run the engine regression tests

Run: `python -m pytest pixel_battle/tests/test_battle_no_lock.py`
`pixel_battle/tests/test_effect_enforcement.py pixel_battle/tests/test_hitstop.py -v`
Expected: all green.

☐ Step 7: Commit

```
git add pixel_battle/engine/battle.py pixel_battle/tests/test_effect_application.py
git commit -m "feat(pixel-battle/engine): skills apply status effects on hit/cast"
```

Task 5: Condition compiler

Files: - Create: `pixel_battle/script/__init__.py` - Create: `pixel_battle/script/conditions.py` -
Test: `pixel_battle/tests/test_script_conditions.py` (new)

☐ Step 1: Write the failing test

```
# pixel_battle/tests/test_script_conditions.py
"""Script `until` condition compiler."""
import pytest

from pixel_battle.engine.character import Character
from pixel_battle.engine.effects import StatusEffect, ROOT
from pixel_battle.script.conditions import (
    compile_condition, ConditionContext, ConditionError,
)

def _ctx(dist=150.0, elapsed=0, attacked=False, hp=100, opp_hp=100,
        opp_effects=None):
    char = Character.load("garen"); char.hp = hp; char.action_state = "idle"
    opp = Character.load("lux"); opp.hp = opp_hp
    opp.effects = list(opp_effects or [])
    return ConditionContext(dist=dist, intent_elapsed_ms=elapsed,
                           char=char, opponent=opp, attacked_this_intent=attacked)

def test_dist_conditions():
    assert compile_condition("dist>=200")(_ctx(dist=250)) is True
    assert compile_condition("dist>=200")(_ctx(dist=150)) is False
    assert compile_condition("dist<=110")(_ctx(dist=90)) is True
```

```

def test_time_condition():
    assert compile_condition("time>=600")(_ctx(elapsed=700)) is True
    assert compile_condition("time>=600")(_ctx(elapsed=500)) is False

def test_skill_done_condition():
    cond = compile_condition("skill_done")
    c_attacking = _ctx(attacked=True); c_attacking.char.action_state = "attacking"
    assert cond(c_attacking) is False           # still attacking
    assert cond(_ctx(attacked=True)) is True     # attacked + now idle
    assert cond(_ctx(attacked=False)) is False   # never attacked

def test_hp_conditions():
    assert compile_condition("hp<=40")(_ctx(hp=30)) is True
    assert compile_condition("target_hp<=40")(_ctx(opp_hp=20)) is True

def test_target_has_condition():
    rooted = [StatusEffect(kind=ROOT, remaining_ms=500)]
    assert compile_condition("target_has:root")(_ctx(opp_effects=rooted)) is True
    assert compile_condition("target_has:root")(_ctx()) is False

def test_bad_conditions_raise():
    with pytest.raises(ConditionError):
        compile_condition("garbage")
    with pytest.raises(ConditionError):
        compile_condition("dist>=notanumber")
    with pytest.raises(ConditionError):
        compile_condition("target_has:nonsense")

```

☐ **Step 2: Run the test to verify it fails**

Run: `python -m pytest pixel_battle/tests/test_script_conditions.py -v` Expected: FAIL —
 ModuleNotFoundError: No module named 'pixel_battle.script'

☐ **Step 3: Create the script package + conditions.py**

```

# pixel_battle/script/__init__.py
"""Scripted-combat package: condition compiler, loader, and ScriptDriver."""

```

```

# pixel_battle/script/conditions.py
"""Compile a script `until` string into a predicate over a ConditionContext."""
from __future__ import annotations
import operator
from dataclasses import dataclass
from typing import Callable

from pixel_battle.engine.effects import EFFECT_KINDS


class ConditionError(ValueError):
    """Raised when an `until` condition string cannot be compiled."""


@dataclass
class ConditionContext:
    """Everything a condition can read about the current tick."""
    dist: float          # horizontal distance between the fighters
    intent_elapsed_ms: int # ms elapsed inside the active intent
    char: object          # the scripted Character
    opponent: object      # the other Character
    attacked_this_intent: bool # did `char` start an attack during this intent


_CMP = {">=": operator.ge, "<=": operator.le}
Predicate = Callable[[ConditionContext], bool]


def compile_condition(src: str) -> Predicate:
    """Compile an `until` string (e.g. 'dist>=230') into a predicate."""
    s = src.strip()

    if s == "skill_done":
        return lambda c: c.attacked_this_intent and \
            c.char.action_state != "attacking"

    if s.startswith("target_has:"):
        effect = s.split(":", 1)[1].strip()
        if effect not in EFFECT_KINDS:
            raise ConditionError(f"unknown effect in condition: {src!r}")
        return lambda c: c.opponent.has_effect(effect)

```

```

for op_s, op_f in _CMP.items():
    if op_s in s:
        field, _, num = s.partition(op_s)
        field = field.strip()
        try:
            n = float(num.strip())
        except ValueError:
            raise ConditionError(f"bad number in condition: {src!r}")
        if field == "dist":
            return lambda c, n=n, f=op_f: f(c.dist, n)
        if field == "time":
            return lambda c, n=n, f=op_f: f(c.intent_elapsed_ms, n)
        if field == "hp":
            return lambda c, n=n, f=op_f: f(c.char.hp, n)
        if field == "target_hp":
            return lambda c, n=n, f=op_f: f(c.opponent.hp, n)
        raise ConditionError(f"unknown field {field!r} in condition: {src!r}")

raise ConditionError(f"unparseable condition: {src!r}")

```

☐ Step 4: Run the tests to verify they pass

Run: `python -m pytest pixel_battle/tests/test_script_conditions.py -v` Expected: PASS (6/6).

☐ Step 5: Commit

```

git add pixel_battle/script/__init__.py pixel_battle/script/conditions.py
    pixel_battle/tests/test_script_conditions.py
git commit -m "feat(pixel-battle/script): until-condition compiler"

```

Task 6: Script loader

Files: - Create: `pixel_battle/script/loader.py` - Test: `pixel_battle/tests/test_script_loader.py` (new)

☐ Step 1: Write the failing test

```

# pixel_battle/tests/test_script_loader.py
"""YAML fight-script loader + validation."""
import textwrap

```

```

import pytest

from pixel_battle.script.loader import load_script_text, ScriptError, FightScript

_GOOD = textwrap.dedent("""
    name: "Test fight"
    left: garen
    right: lux
    left_script:
        - {do: advance, until: "dist<=110"}
        - {do: "attack:basic", until: skill_done}
    right_script:
        - {do: retreat, until: "dist>=230"}
        - {do: "attack:cd", until: skill_done}
""")

def test_loads_a_valid_script():
    s = load_script_text(_GOOD)
    assert isinstance(s, FightScript)
    assert s.left == "garen" and s.right == "lux"
    assert len(s.left_intents) == 2
    assert s.left_intents[0].do == "advance"
    assert s.right_intents[1].do == "attack:cd"
    # `until` is compiled to a callable predicate.
    assert callable(s.left_intents[0].until)

def test_rejects_unknown_character():
    bad = _GOOD.replace("left: garen", "left: nobody")
    with pytest.raises(ScriptError):
        load_script_text(bad)

def test_rejects_unknown_do_verb():
    bad = _GOOD.replace("do: advance", "do: teleport")
    with pytest.raises(ScriptError):
        load_script_text(bad)

def test_rejects_bad_condition():

```

```

bad = _GOOD.replace('until: "dist<=110"', 'until: "garbage"')
with pytest.raises(ScriptError):
    load_script_text(bad)

def test_rejects_missing_field():
    bad = _GOOD.replace("right: lux\n", "")
    with pytest.raises(ScriptError):
        load_script_text(bad)

```

☐ Step 2: Run the test to verify it fails

Run: `python -m pytest pixel_battle/tests/test_script_loader.py -v` Expected: FAIL — `pixel_battle.script.loader` does not exist.

☐ Step 3: Create `loader.py`

```

# pixel_battle/script/loader.py
"""Load + validate a YAML fight script into a FightScript."""
from __future__ import annotations
from dataclasses import dataclass
from pathlib import Path
from typing import List

import yaml

from pixel_battle.engine.character import DATA_PATH
from pixel_battle.script.conditions import (
    compile_condition, ConditionError, Predicate,
)

# `do` verbs the driver understands → engine action integers.
DO_VERBS = {
    "idle": 0, "retreat": 1, "advance": 2, "jump": 3,
    "attack:basic": 4, "attack:cd": 5, "attack:ultimate": 6,
    "attack:special": 7, "attack:kick": 8,
}

class ScriptError(ValueError):
    """Raised when a fight script is malformed."""

```

```

@dataclass
class Intent:
    do: str          # a key of DO_VERBS
    until: Predicate  # compiled condition
    until_src: str    # the raw condition string (for debugging)

```

```

@dataclass
class FightScript:
    name: str
    left: str          # left character id
    right: str          # right character id
    left_intents: List[Intent]
    right_intents: List[Intent]

```

```

def _known_character_ids() -> set:
    import json
    with open(DATA_PATH, encoding="utf-8") as f:
        return set(json.load(f).keys())

```

```

def _parse_intents(raw, side: str) -> List[Intent]:
    if not isinstance(raw, list) or not raw:
        raise ScriptError(f"{side}_script must be a non-empty list")
    intents = []
    for i, item in enumerate(raw):
        if not isinstance(item, dict) or "do" not in item or "until" not in item:
            raise ScriptError(f"{side}_script[{i}] needs 'do' and 'until'")
        do = str(item["do"])
        if do not in DO_VERBS:
            raise ScriptError(f"{side}_script[{i}]: unknown do verb {do!r}")
        until_src = str(item["until"])
        try:
            until = compile_condition(until_src)
        except ConditionError as e:
            raise ScriptError(f"{side}_script[{i}]: {e}") from e
        intents.append(Intent(do=do, until=until, until_src=until_src))
    return intents

```

```

def load_script_text(text: str) -> FightScript:
    """Parse + validate a fight script from YAML text."""
    try:
        data = yaml.safe_load(text)
    except yaml.YAMLError as e:
        raise ScriptError(f"invalid YAML: {e}") from e
    if not isinstance(data, dict):
        raise ScriptError("script must be a YAML mapping")
    for key in ("name", "left", "right", "left_script", "right_script"):
        if key not in data:
            raise ScriptError(f"script missing required key: {key!r}")

    known = _known_character_ids()
    for side in ("left", "right"):
        if data[side] not in known:
            raise ScriptError(f"unknown character id: {data[side]!r}")

    return FightScript(
        name=str(data["name"]),
        left=str(data["left"]), right=str(data["right"]),
        left_intents=_parse_intents(data["left_script"], "left"),
        right_intents=_parse_intents(data["right_script"], "right"),
    )

def load_script(path) -> FightScript:
    """Load + validate a fight script from a YAML file."""
    return load_script_text(Path(path).read_text(encoding="utf-8"))

```

Note: `DATA_PATH` is the `characters.json` path already defined in `character.py` — confirm that name when implementing; if it is not exported, use `pixel_battle/data/characters.json` resolved relative to the package root.

☐ Step 4: Run the tests to verify they pass

Run: `python -m pytest pixel_battle/tests/test_script_loader.py -v` Expected: PASS (5/5).

☐ Step 5: Commit

```

git add pixel_battle/script/loader.py pixel_battle/tests/test_script_loader.py
git commit -m "feat(pixel-battle/script): YAML fight-script loader + validation"

```

Task 7: ScriptDriver

Files: - Create: `pixel_battle/script/driver.py` - Test: `pixel_battle/tests/test_script_driver.py` (new)

☐ Step 1: Write the failing test

```
# pixel_battle/tests/test_script_driver.py
"""ScriptDriver – per-tick intent tracking + action emission."""
import os
os.environ.setdefault("SDL_VIDEODRIVER", "dummy")

from pixel_battle.rl.env import PixelBattleEnv
from pixel_battle.script.loader import load_script_text
from pixel_battle.script.driver import ScriptDriver, INTENT_MAX_MS

_SCRIPT = """
name: "Driver test"
left: garen
right: lux
left_script:
  - {do: advance, until: "dist<=110"}
  - {do: idle, until: "time>=100000"}
right_script:
  - {do: retreat, until: "dist>=100000"}
  - {do: idle, until: "time>=100000"}
"""

def _env_driver():
    env = PixelBattleEnv(seed=1)
    driver = ScriptDriver(load_script_text(_SCRIPT))
    return env, driver

def test_driver_emits_intent_actions():
    env, driver = _env_driver()
    env.left.pos_x, env.right.pos_x = 240.0, 460.0 # far apart
    left_act, right_act = driver.decide(env.battle)
    assert left_act == 2 # advance
    assert right_act == 1 # retreat
```

```

def test_driver_advances_intent_when_condition_met():
    env, driver = _env_driver()
    env.left.pos_x, env.right.pos_x = 240.0, 300.0    # dist 60 <= 110
    driver.decide(env.battle)                          # left's intent-0 until met
    left_act, _ = driver.decide(env.battle)
    assert left_act == 0    # advanced to intent-1 (idle)

def test_driver_timeout_advances_a_stuck_intent():
    env, driver = _env_driver()
    env.left.pos_x, env.right.pos_x = 240.0, 460.0    # dist never <= 110
    # Pump more than INTENT_MAX_MS of ticks; the stuck intent must advance.
    ticks = INTENT_MAX_MS // 16 + 5
    for _ in range(ticks):
        driver.decide(env.battle)
        env.battle.elapsed_ms += 16
    left_act, _ = driver.decide(env.battle)
    assert left_act == 0    # timed out of advance → now on intent-1 (idle)

def test_exhausted_script_idles():
    script = """
name: "short"
left: garen
right: lux
left_script:
  - {do: advance, until: "time>=0"}
right_script:
  - {do: idle, until: "time>=0"}
"""

    env = PixelBattleEnv(seed=1)
    driver = ScriptDriver(load_script_text(script))
    for _ in range(5):
        left_act, _ = driver.decide(env.battle)
        env.battle.elapsed_ms += 16
    assert left_act == 0    # ran out of intents → idle

```

☐ Step 2: Run the test to verify it fails

Run: `python -m pytest pixel_battle/tests/test_script_driver.py -v` Expected: FAIL — `pixel_battle.script.driver` does not exist.

☐ Step 3: Create `driver.py`

```
# pixel_battle/script/driver.py
"""ScriptDriver – turns a FightScript into per-tick engine actions."""
from __future__ import annotations

from pixel_battle.script.conditions import ConditionContext
from pixel_battle.script.loader import DO_VERBS, FightScript, Intent

# An intent that never satisfies its `until` is force-advanced after this long,
# so a script can never hang.
INTENT_MAX_MS = 4000


class _SideState:
    """Per-character cursor through an intent list."""
    def __init__(self, intents):
        self.intents = intents
        self.index = 0
        self.intent_start_ms = None # battle.elapsed_ms when intent began
        self.attacked_this_intent = False

    def active(self):
        if self.index < len(self.intents):
            return self.intents[self.index]
        return None


class ScriptDriver:
    """Plays a FightScript: each tick, emits (left_action, right_action)."""

    def __init__(self, script: FightScript):
        self.script = script
        self._left = _SideState(script.left_intents)
        self._right = _SideState(script.right_intents)

    def decide(self, battle) -> tuple:
        """Return (left_action, right_action) for the current battle tick."""
        dist = abs(battle.left.pos_x - battle.right.pos_x)
```

```

left_act = self._decide_side(self._left, battle.left, battle.right,
                             dist, battle.elapsed_ms)
right_act = self._decide_side(self._right, battle.right, battle.left,
                              dist, battle.elapsed_ms)

return left_act, right_act

def _decide_side(self, state: _SideState, char, opp,
                dist: float, now_ms: int) -> int:
    intent = state.active()
    if intent is None:
        return DO_VERBS["idle"]          # script exhausted -> idle

    if state.intent_start_ms is None:
        state.intent_start_ms = now_ms
        state.attacked_this_intent = False

    elapsed = now_ms - state.intent_start_ms
    if char.action_state == "attacking":
        state.attacked_this_intent = True

    ctx = ConditionContext(
        dist=dist, intent_elapsed_ms=elapsed, char=char, opponent=opp,
        attacked_this_intent=state.attacked_this_intent)

    if intent.until(ctx) or elapsed >= INTENT_MAX_MS:
        state.index += 1
        state.intent_start_ms = None      # next intent starts fresh
        intent = state.active()
        if intent is None:
            return DO_VERBS["idle"]

    return DO_VERBS[intent.do]

```

☐ **Step 4: Run the tests to verify they pass**

Run: `python -m pytest pixel_battle/tests/test_script_driver.py -v` Expected: PASS (4/4).

☐ **Step 5: Commit**

```

git add pixel_battle/script/driver.py pixel_battle/tests/test_script_driver.py
git commit -m "feat(pixel-battle/script): ScriptDriver - intent tracking + action
            emission"

```

Task 8: Render integration — action-source abstraction + scripted entry point

Files: - Modify: `pixel_battle/rl/play.py` - Create: `pixel_battle/rl/play_scripted.py` - Test: `pixel_battle/tests/test_play_scripted.py` (new)

`play.py` current facts (verified): `_render_fight(recorder, model, env, max_seconds, end_hold_frames=0)` at line 402; its loop calls `model.predict(obs_left, deterministic=False)` / `model.predict(obs_right, ...)` (lines 444-445) then `env.step((int(left_act), int(right_act)))` (line 448). `run_one_match` (line 723) and `run_full_episode` (line 767) call `_render_fight` passing `model`.

☐ Step 1: Write the failing test

```
# pixel_battle/tests/test_play_scripted.py
"""Scripted render path – action-source abstraction + ScriptDriver wiring."""
import os
os.environ.setdefault("SDL_VIDEODRIVER", "dummy")

def test_rl_action_source_is_callable():
    from pixel_battle.rl.play import _rl_action_source

    class _FakeModel:
        def predict(self, obs, deterministic=False):
            return 0, None

    src = _rl_action_source(_FakeModel())
    assert callable(src)

def test_script_action_source_drives_from_a_script():
    from pixel_battle.rl.env import PixelBattleEnv
    from pixel_battle.rl.play_scripted import _script_action_source
    from pixel_battle.script.driver import ScriptDriver
    from pixel_battle.script.loader import load_script_text

    script = """
name: "src test"
left: garen
right: lux
left_script:
```

```

- {do: advance, until: "time>=100000"}
right_script:
- {do: retreat, until: "time>=100000"}
"""

env = PixelBattleEnv(seed=1)
env.reset()
env.left.pos_x, env.right.pos_x = 240.0, 400.0
src = _script_action_source(ScriptDriver(load_script_text(script)))
left_act, right_act = src(env, None)
assert left_act == 2 and right_act == 1    # advance / retreat

```

☐ Step 2: Run the test to verify it fails

Run: `python -m pytest pixel_battle/tests/test_play_scripted.py -v` Expected: FAIL — `_rl_action_source` / `play_scripted` do not exist.

☐ Step 3: Generalise `_render_fight` to an action source

In `pixel_battle/rl/play.py`:

1. Change the `_render_fight` signature — rename the `model` parameter to `action_source`:

```

def _render_fight(recorder: FrameRecorder, action_source, env,
                  max_seconds: float, end_hold_frames: int = 0) -> dict:

```

2. Replace the two `model.predict(...)` lines (444-445) with a single call:

```

left_act, right_act = action_source(env, (obs_left, obs_right))

```

Leave the `env.step((int(left_act), int(right_act)))` call as-is.

3. Add a helper near `_render_fight` that wraps a PPO model as an action source:

```

def _rl_action_source(model):
    """Adapt a PPO model to the (env, obs) -> (left_act, right_act) interface."""
    def _source(env, obs):
        obs_left, obs_right = obs
        left_act, _ = model.predict(obs_left, deterministic=False)
        right_act, _ = model.predict(obs_right, deterministic=False)
        return int(left_act), int(right_act)
    return _source

```

4. Update the two callers — `run_one_match` and `run_full_episode` — to wrap their `model` argument: change `_render_fight(recorder, model, env, ...)` to `_render_fight(recorder, _rl_action_source(model), env, ...)` in both.

□ **Step 4: Create `play_scripted.py`**

```
# pixel_battle/rl/play_scripted.py
"""Render a fight from an authored YAML script (no RL policy)."""
from __future__ import annotations
import argparse
import os
from pathlib import Path

os.environ.setdefault("SDL_VIDEODRIVER", "dummy")
import pygame # noqa: E402

from pixel_battle.rl.env import PixelBattleEnv # noqa: E402
from pixel_battle.rl.play import _render_fight, FrameRecorder, WIDTH, HEIGHT, FPS, ROOT
    # noqa: E402
from pixel_battle.script.driver import ScriptDriver # noqa: E402
from pixel_battle.script.loader import load_script # noqa: E402

OUT_DIR = ROOT / "pixel_battle" / "output" / "scripted"

def _script_action_source(driver: ScriptDriver):
    """Adapt a ScriptDriver to the (env, obs) -> (left_act, right_act) interface."""
    def _source(env, obs):
        return driver.decide(env.battle)
    return _source

def render_script(script_path: Path, out_dir: Path = OUT_DIR) -> Path:
    """Render the fight described by `script_path`; return the mp4 path."""
    pygame.init()
    pygame.display.set_mode((1, 1))
    out_dir.mkdir(parents=True, exist_ok=True)

    script = load_script(script_path)
    env = PixelBattleEnv(left_id=script.left, right_id=script.right)
    driver = ScriptDriver(script)

    raw = out_dir / f"{script_path.stem}_raw.mp4"
```

```

recorder = FrameRecorder(str(raw), fps=FPS, width=WIDTH, height=HEIGHT)
_render_fight(recorder, _script_action_source(driver), env,
               max_seconds=60.0, end_hold_frames=120)
recorder.close()
print(f"Scripted render: {raw}")
return raw

if __name__ == "__main__":
    p = argparse.ArgumentParser()
    p.add_argument("script", type=Path, help="path to a fight-script YAML")
    args = p.parse_args()
    render_script(args.script)

```

Note: `PixelBattleEnv(left_id=..., right_id=...)` — confirm the env’s matchup constructor signature when implementing (the env “supports arbitrary character matchups”; `run_full_episode` in `play.py` already builds a specific matchup — mirror exactly what it does). `FrameRecorder`, `WIDTH`, `HEIGHT`, `FPS`, `R00T` are imported from `play.py` — confirm those names are module-level there; if `FrameRecorder` lives elsewhere, import it from its real module. The `_render_fight` call here uses only the raw recorder; full intro/outro composition (as `run_full_episode` does) can be added later — a raw fight mp4 is sufficient for validation.

☐ Step 5: Run the tests to verify they pass

Run: `python -m pytest pixel_battle/tests/test_play_scripted.py -v` Expected: PASS (2/2).

☐ Step 6: Run the `play.py` regression tests

Run: `python -m pytest pixel_battle/tests/test_play_richness.py pixel_battle/tests/test_skill_vfx.py pixel_battle/tests/test_play_multi_imports.py -v`
 Expected: all green — `_render_fight`’s caller-facing behaviour is unchanged (the RL path still works through `_rl_action_source`).

☐ Step 7: Commit

```

git add pixel_battle/rl/play.py pixel_battle/rl/play_scripted.py
      pixel_battle/tests/test_play_scripted.py
git commit -m "feat(pixel-battle/rl): action-source abstraction + scripted render path"

```

Task 9: Status-effect VFX indicators

Files: - Modify: `pixel_battle/rl/stick_renderer.py` - Test:
`pixel_battle/tests/test_effect_vfx.py` (new)

A small visual indicator per active effect, drawn on the affected character so a viewer can read root / slow / shield / tenacity.

☐ Step 1: Write the failing test

```
# pixel_battle/tests/test_effect_vfx.py
"""Status-effect visual indicators."""
import os
os.environ.setdefault("SDL_VIDEODRIVER", "dummy")

import numpy as np
import pygame
import pytest

from pixel_battle.engine.character import Character
from pixel_battle.engine.effects import StatusEffect, ROOT, SHIELD
from pixel_battle.rl.stick_renderer import draw_effect_indicators

@pytest.fixture(scope="module", autouse=True)
def _pg():
    pygame.init()
    pygame.display.set_mode((1, 1))
    yield
    pygame.quit()

def _nonbg(surf):
    arr = pygame.surfarray.array3d(surf)
    return int(np.any(arr != 0, axis=-1).sum())

def test_no_indicator_when_no_effects():
    c = Character.load("garen")
    c.pos_x, c.pos_y = 240.0, 500.0
    surf = pygame.Surface((480, 854)); surf.fill((0, 0, 0))
    draw_effect_indicators(surf, c)
```

```
assert _nonbg(surf) == 0
```

```
def test_indicator_drawn_for_active_effect():
    c = Character.load("garen")
    c.pos_x, c.pos_y = 240.0, 500.0
    c.effects.append(StatusEffect(kind=ROOT, remaining_ms=1000))
    surf = pygame.Surface((480, 854)); surf.fill((0, 0, 0))
    draw_effect_indicators(surf, c)
    assert _nonbg(surf) > 0

def test_more_effects_draw_more_pixels():
    c = Character.load("garen")
    c.pos_x, c.pos_y = 240.0, 500.0
    c.effects.append(StatusEffect(kind=ROOT, remaining_ms=1000))
    one = pygame.Surface((480, 854)); one.fill((0, 0, 0))
    draw_effect_indicators(one, c)
    c.effects.append(StatusEffect(kind=SHIELD, remaining_ms=1000, magnitude=20))
    two = pygame.Surface((480, 854)); two.fill((0, 0, 0))
    draw_effect_indicators(two, c)
    assert _nonbg(two) > _nonbg(one)
```

☐ Step 2: Run the test to verify it fails

Run: `python -m pytest pixel_battle/tests/test_effect_vfx.py -v` Expected: FAIL —
`draw_effect_indicators` does not exist.

☐ Step 3: Add `draw_effect_indicators` to `stick_renderer.py`

Add this function to `pixel_battle/rl/stick_renderer.py` (alongside the other VFX helpers like `spawn_impact_burst`):

```
# Per-effect indicator colours.
_EFFECT_COLORS = {
    "root": (150, 110, 60), # shackle brown
    "slow": (90, 140, 235), # cold blue
    "shield": (235, 225, 110), # golden
    "tenacity": (210, 120, 220), # violet
}
```

```
def draw_effect_indicators(surf, char) -> None:
```

```

"""Draw a small dot per active status effect in a row above the head."""
effects = getattr(char, "effects", None)
if not effects:
    return
cx = int(char.pos_x)
top_y = int(char.pos_y) - 210 # above the figure
spacing = 16
n = len(effects)
start_x = cx - (n - 1) * spacing // 2
for i, effect in enumerate(effects):
    color = _EFFECT_COLORS.get(effect.kind, (220, 220, 220))
    ex = start_x + i * spacing
    pygame.draw.circle(surf, color, (ex, top_y), 6)
    pygame.draw.circle(surf, (0, 0, 0), (ex, top_y), 6, 2)

```

☐ Step 4: Call it from the render loop

In `pixel_battle/rl/play.py`'s `_render_fight`, where the two characters are drawn each frame (the `draw_stick_figure(world, env.left, ...)` / `draw_stick_figure(world, env.right, ...)` calls), add right after them:

```

draw_effect_indicators(world, env.left)
draw_effect_indicators(world, env.right)

```

Add `draw_effect_indicators` to the existing `from pixel_battle.rl.stick_renderer import (...)` import in `play.py`.

☐ Step 5: Run the tests + regression

Run: `python -m pytest pixel_battle/tests/test_effect_vfx.py pixel_battle/tests/test_stick_renderer_pose.py pixel_battle/tests/test_play_richness.py -v`
Expected: all green.

☐ Step 6: Commit

```

git add pixel_battle/rl/stick_renderer.py pixel_battle/rl/play.py
      pixel_battle/tests/test_effect_vfx.py
git commit -m "feat(pixel-battle/rl): status-effect VFX indicators"

```

Task 10: Starter script library

Files: - Create: `pixel_battle/data/scripts/01_lux_kite_garen.yaml` (+ 4 more) - Test: `pixel_battle/tests/test_script_library.py` (new)

Author **5** fight scripts. Below is one complete exemplar; author the other 4 following the same structure for these matchups/themes:

- `01_lux_kite_garen.yaml` — Lux kites Garen, roots, casts (the exemplar below).
- `02_garen_rush_lux.yaml` — Garen rushes down Lux, `courage` (tenacity) to power through, ultimate finish.
- `03_glass_slow_brick.yaml` — Glass Slab uses `shard_scatter` (slow) to control Brick Phone, kites and chips.
- `04_yasuo_duel_ashe.yaml` — Yasuo closes on Ashe with dashes, trades, comeback at low HP.
- `05_lux_barrier_yasuo.yaml` — Lux casts `prismatic_barrier` (shield) to survive Yasuo's burst, then roots and wins.

Each must be a valid script per Task 6's loader. Use `attack:cd` for cooldown skills (e.g. Lux's `light_binding` root, Glass's `shard_scatter` slow), `attack:special` for specials (Lux's `prismatic_barrier` shield, Garen's `courage` tenacity), `attack:basic` / `attack:ultimate` as appropriate. Pace intents with `dist`, `skill_done`, `hp`, and `target_has` conditions.

☐ Step 1: Write the exemplar script

```
# pixel_battle/data/scripts/01_lux_kite_garen.yaml
name: "Lux kites and roots Garen"
left: garen
right: lux
left_script:
  - {do: advance,          until: "dist<=130"}
  - {do: "attack:basic",   until: skill_done}
  - {do: advance,          until: "dist<=130"}
  - {do: "attack:basic",   until: skill_done}
  - {do: advance,          until: "dist<=110"}
  - {do: "attack:cd",      until: skill_done}
  - {do: advance,          until: "dist<=110"}
  - {do: "attack:ultimate", until: skill_done}
  - {do: advance,          until: "dist<=120"}
  - {do: "attack:basic",   until: skill_done}
right_script:
  - {do: retreat,          until: "dist>=235"}
  - {do: "attack:cd",      until: skill_done}      # light_binding – roots Garen
  - {do: "attack:special", until: skill_done}      # lucent_singularity
```

```

- {do: retreat,          until: "dist>=220"}
- {do: "attack:special", until: skill_done}
- {do: retreat,          until: "dist>=235"}
- {do: "attack:cd",      until: skill_done}
- {do: "attack:special", until: skill_done}
- {do: idle,             until: "time>=400"}
- {do: "attack:basic",   until: skill_done}

```

☐ Step 2: Write the other 4 scripts

Create `02_garen_rush_lux.yaml`, `03_glass_slow_brick.yaml`, `04_yasuo_duel_ashe.yaml`, `05_lux_barrier_yasuo.yaml` in `pixel_battle/data/scripts/`, each following the exemplar's structure and its matchup/theme from the list above. Each is two intent sequences of ~8-12 intents using only the `do` verbs in `D0_VERBS` and the conditions Task 5 supports.

☐ Step 3: Write the library test

```

# pixel_battle/tests/test_script_library.py
"""Every shipped fight script loads + validates."""
from pathlib import Path

from pixel_battle.script.loader import load_script

_SCRIPT_DIR = Path(__file__).resolve().parents[1] / "data" / "scripts"

def test_all_scripts_load():
    scripts = sorted(_SCRIPT_DIR.glob("*.yaml"))
    assert len(scripts) >= 5, "expected at least 5 starter scripts"
    for path in scripts:
        s = load_script(path) # raises ScriptError if invalid
        assert s.left and s.right
        assert s.left_intents and s.right_intents

```

☐ Step 4: Run the test to verify it passes

Run: `python -m pytest pixel_battle/tests/test_script_library.py -v` Expected: PASS — all 5 scripts load and validate. Fix any script the loader rejects.

☐ Step 5: Commit

```
git add pixel_battle/data/scripts/ pixel_battle/tests/test_script_library.py
```

```
git commit -m "feat(pixel-battle): starter fight-script library (5 scripts)"
```

Task 11: Part 0 — renderer micro-polish

Files: - Modify: `pixel_battle/rl/play.py` - Modify: `pixel_battle/rl/poses.py` - Modify: `pixel_battle/tests/test_poses.py` (stale comment only, if present)

☐ Step 1: Smaller characters — lower `CAM_ZOOM`

In `pixel_battle/rl/play.py`, change `CAM_ZOOM` from 1.45 to 1.2:

```
CAM_ZOOM = 1.2           # was 1.45 — smaller fighters, near the original scale
```

If `pixel_battle/tests/test_poses.py` has a comment quoting the old `CAM_VIEW_H` derived from `CAM_ZOOM = 1.45`, update it for 1.2 ($854 / 1.2 \approx 712$ tall). The `_MAX_HEIGHT` / `_MAX_HALF_W` constants stay — a lower zoom only adds headroom.

☐ Step 2: Smoother animation — pose interpolation spans the engine phase

In `pixel_battle/rl/poses.py`, the renderer's `_PHASE_DUR` table holds per-archetype windup/strike/recover durations that are **shorter** than the engine's real attack-phase durations (`ATTACK_WINDUP_MS = 200`, `ATTACK_ACTIVE_MS = 90`, `ATTACK_RECOVER_MS = 250` in `battle.py`), so each motion finishes early and then holds. Make the interpolation span the engine's real phases: set every archetype's `_PHASE_DUR` entry to `{"windup": 200, "strike": 90, "recover": 250}` (matching the engine), so the pose interpolates across every frame of the phase.

Concretely — replace the per-archetype `_PHASE_DUR` values with the engine-matched durations, and the `_DEFAULT_PHASE_DUR` likewise:

```
_ENGINE_PHASE = {"windup": 200, "strike": 90, "recover": 250}
_PHASE_DUR = {a: dict(_ENGINE_PHASE) for a in (
    "melee", "slam", "spin", "dash", "bolt", "multishot", "aura", "beam", "kick")}
_DEFAULT_PHASE_DUR = dict(_ENGINE_PHASE)
```

☐ Step 3: Run the full pose + render test suite

Run: `python -m pytest pixel_battle/tests/test_poses.py`

`pixel_battle/tests/test_stick_renderer_pose.py pixel_battle/tests/test_play_richness.py -v`

Expected: all green — the visual-safety lock still passes (more camera headroom; the pose *targets* are unchanged, only the interpolation pacing changed).

☐ Step 4: Commit

```
git add pixel_battle/rl/play.py pixel_battle/rl/poses.py pixel_battle/tests/test_poses.py
git commit -m "feat(pixel-battle/rl): smaller framing + smoother pose interpolation"
```

Task 12: Full test sweep + scripted validation render

Files: none (verification only; possible tuning commits)

☐ **Step 1: Run the entire test suite**

Run: `python -m pytest pixel_battle/tests/ -q` Expected: all green. Fix any regression before continuing.

☐ **Step 2: Render every starter script**

For each script in `pixel_battle/data/scripts/`, run: `python -m pixel_battle.rl.play_scripted pixel_battle/data/scripts/<name>.yaml` Expected: each produces an mp4 in `pixel_battle/output/scripted/` with no Python error/traceback. If a render crashes, that is a real bug — diagnose and fix.

☐ **Step 3: Inspect the output**

For each rendered mp4, `ffprobe -v error -show_entries format=duration,size -of default=noprint_wrappers=1 <file>` and confirm a sane duration. Watch them: each fight should play its authored choreography — the mage kites and roots; root / slow / shield / tenacity indicators read clearly; characters are smaller and motion is smoother.

☐ **Step 4: Tune**

Adjust from what the renders show, then re-render: - A fight reads wrong / too fast / too slow → adjust that script's intent conditions in its YAML. - Effect too strong / weak / short → adjust the `applies` duration/magnitude in `characters.json`. - Characters still too large → adjust `CAM_ZOOM` in `play.py`. After any change, re-run `python -m pytest pixel_battle/tests/ -q`.

☐ **Step 5: Commit any tuning changes**

```
git add -A
git commit -m "tune(pixel-battle): scripted-combat validation tuning"
```

Self-Review

Spec coverage — every spec section maps to a task: - §5 ScriptDriver as action source → Tasks 7, 8 - §6 script format (intent sequence, `do` verbs, `until` conditions, robustness timeout) → Tasks 5 (conditions), 6 (loader/format), 7 (driver + `INTENT_MAX_MS`) - §7 status-effect system (4 effects, `applies`) → Tasks 1 (`applies` /data), 2 (model + shield), 3 (lifecycle + root/slow), 4 (application + tenacity); VFX → Task 9 - §8 Part 0 (smaller + smoother) → Task 11 - §9 error handling → Task 6 (loader rejects malformed scripts), Task 7 (`INTENT_MAX_MS` no-hang), Task 1 (`SkillApplies` rejects unknown effect) - §10 testing → every task is TDD; Task 12 runs the full sweep - §11 validation → Task 12 - §3 no retrain → no task touches the RL observation/action space/reward; `_render_fight` 's RL path is preserved via `_rl_action_source`

No placeholders — Task 10 ships one complete exemplar script and an explicit list of 4 more concrete matchups/themes (scripts are tunable content; Task 12 is the tuning pass). All constants are concrete.

Type consistency — checked: `StatusEffect(kind, remaining_ms, magnitude)` / `SkillApplies(effect, duration_ms, magnitude, target)` (Tasks 1-4); `Character.effects` / `effect_of` / `has_effect` (Task 2) used in Tasks 3, 4, 5, 9; `EFFECT_KINDS` (Task 1) used in Task 5; `ConditionContext` fields (Task 5) match `ScriptDriver` 's construction of it (Task 7); `Intent` / `FightScript` / `D0_VERBS` (Task 6) used by Task 7 and Task 8; `compile_condition` / `ConditionError` (Task 5) used by Task 6; `_render_fight(recorder, action_source, env, ...)` (Task 8) and `_rl_action_source` / `_script_action_source` consistent; `draw_effect_indicators` (Task 9).

Known verification points for the implementer (flagged inline, not placeholders): `DATA_PATH` export name in `character.py` (Task 6); the `PixelBattleEnv` matchup constructor signature and the `FrameRecorder` / `R00T` import names in `play.py` (Task 8) — each task says to confirm against the real code and mirror the existing pattern.